

DOCKER

How containers actually work — the illustrated field guide



with Cargo, your guide

Part of the Interview-Preparation illustrated series

Table of Contents

1. A Container Is Not a Small VM
2. Anatomy of an Image: The Layer Cake
3. Dockerfile Order Controls Your Cache
4. Multi-Stage Builds: Travel Light
5. The Container Lifecycle
6. docker stop vs. docker kill
7. Choosing a Network Mode
8. Volumes Outlive Containers
9. Copy-on-Write: Sharing Without Sharing
10. Cheat Sheet

This short guide is meant to build *intuition* — the mental pictures that make the detailed reference material (like the companion 150-question Docker Q&A file) click into place. It won't replace hands-on practice, but it should make the concepts stick a lot faster.

CHAPTER 1

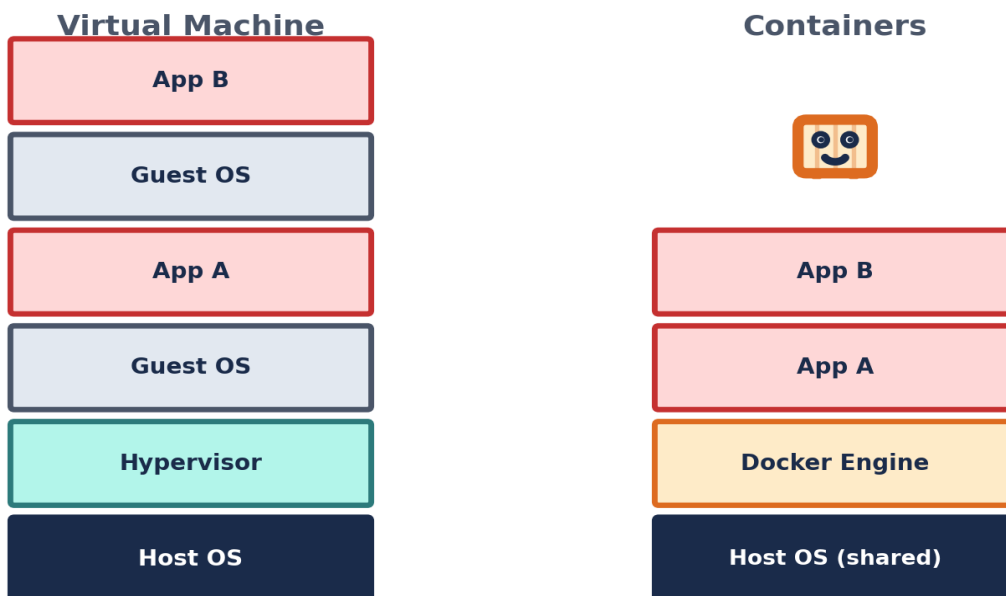
A Container Is Not a Small VM

The single most common misconception about Docker is thinking of a container as a lightweight virtual machine. It isn't — and the difference explains almost every performance and security property containers have.

A virtual machine virtualizes an entire computer, including its own guest operating system kernel, running on top of a hypervisor. Every VM drags along a full copy of an OS — megabytes (often gigabytes) of overhead before your application even starts.

A container shares the host machine's kernel. Docker uses two Linux kernel features — **namespaces** (which give a process its own isolated view of the process list, network interfaces, and filesystem) and **cgroups** (which cap how much CPU/memory a process can use) — to make a process *believe* it has its own machine, without actually duplicating the OS.

Virtual Machine vs. Container



Containers share the host kernel: lighter, faster to start, no duplicate OS.

IN SHORT: Containers isolate processes using the host's own kernel (namespaces + cgroups). VMs isolate entire machines using a hypervisor. That's why containers start in milliseconds and VMs take seconds to minutes.

CHAPTER 2

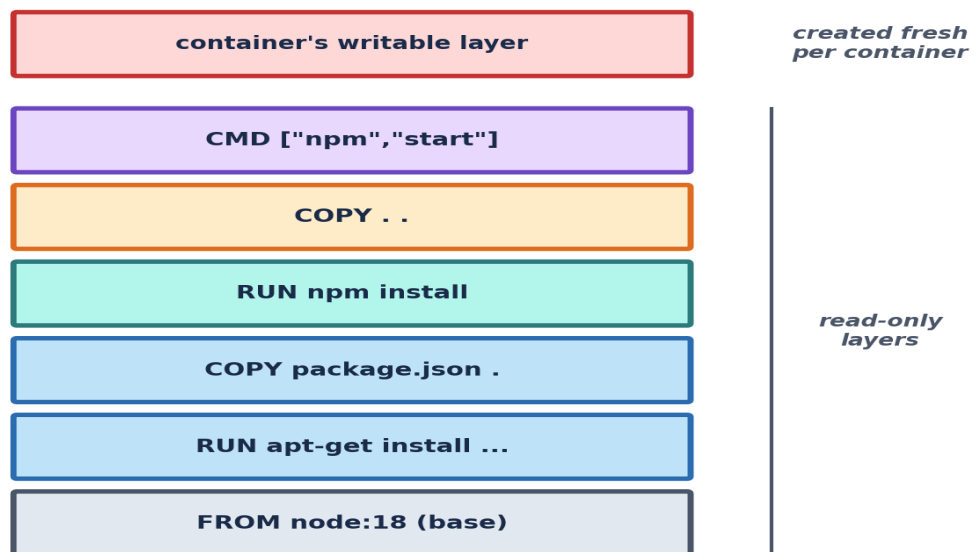
Anatomy of an Image: The Layer Cake

Every instruction in a Dockerfile that touches the filesystem — RUN, COPY, ADD — creates a new, immutable, read-only layer. Docker stacks these layers using a union filesystem, presenting them to the container as one single, seamless filesystem.

When you actually run a container, Docker adds one more layer on top: a thin, **writable** layer unique to that specific container. Any file the container creates or modifies at runtime goes here — never into the shared, read-only layers beneath it.

This is also why the same image can be safely used by a thousand different running containers simultaneously: they're all reading from the exact same shared, read-only layers, and each only writes to its own private layer on top.

An Image Is a Stack of Layers



IN SHORT: An image = a stack of read-only layers. A container = that same stack, plus one thin writable layer just for that instance. Delete the container, and only that top writable layer disappears.

CHAPTER 3

Dockerfile Order Controls Your Cache

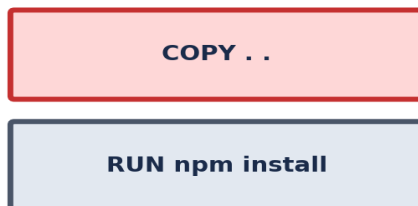
Docker caches each layer, and reuses that cache on your next build as long as the instruction — and everything before it — hasn't changed. The moment one instruction's cache is invalidated, **every single instruction after it** must be rebuilt from scratch too, even if those later steps didn't actually change.

This is why instruction order matters so much. If you `COPY . .` (your entire source tree) *before* installing dependencies, then every single source-code edit — even a typo fix in a comment — invalidates the dependency-install layer, forcing a full, slow reinstall on every build.

The fix is almost always the same: copy just the dependency manifest first (package.json, pom.xml, requirements.txt), install dependencies, *then* copy the rest of your source code. Now the expensive install step only re-runs when your actual dependencies change.

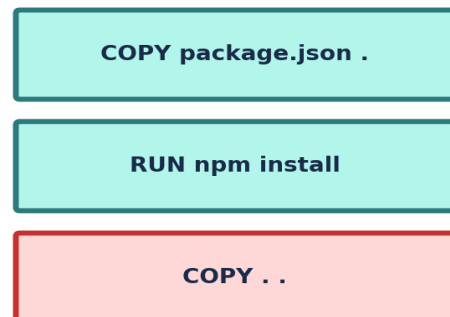
Dockerfile Order Controls Cache Reuse

SLOW rebuilds



*Any source change invalidates
BOTH layers -- reinstalls deps
every single build*

FAST rebuilds



*Dependency layer only rebuilds
when package.json actually changes*



IN SHORT: Order your Dockerfile from least-frequently-changing to most-frequently-changing. Dependencies first, source code last — that's the single highest-leverage Docker habit you can build.

CHAPTER 4

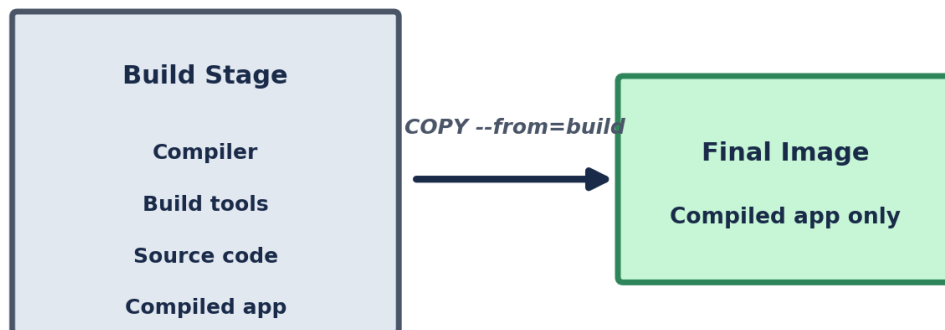
Multi-Stage Builds: Travel Light

Compiling an application often needs a heavyweight toolchain — a compiler, build tools, the full source tree — none of which your application actually needs to *run*. If you build and run in a single stage, all of that bulk ships in your final image too.

A multi-stage Dockerfile uses multiple FROM statements. An early stage does the heavy lifting (compiling, building), and the *final* stage starts fresh from a minimal base image, copying over only the specific compiled artifact it actually needs via `COPY --from=build`.

The result: a final image that might be tens of megabytes instead of over a gigabyte — and, as a security bonus, no compiler or build tools sitting around inside your production image for an attacker to potentially misuse.

Multi-Stage Builds: Travel Light



Build tools never make it into the final, deployed image.

IN SHORT: Use one stage to build, a separate minimal stage to run. Copy across only the finished artifact — leave the toolchain behind.

CHAPTER 5

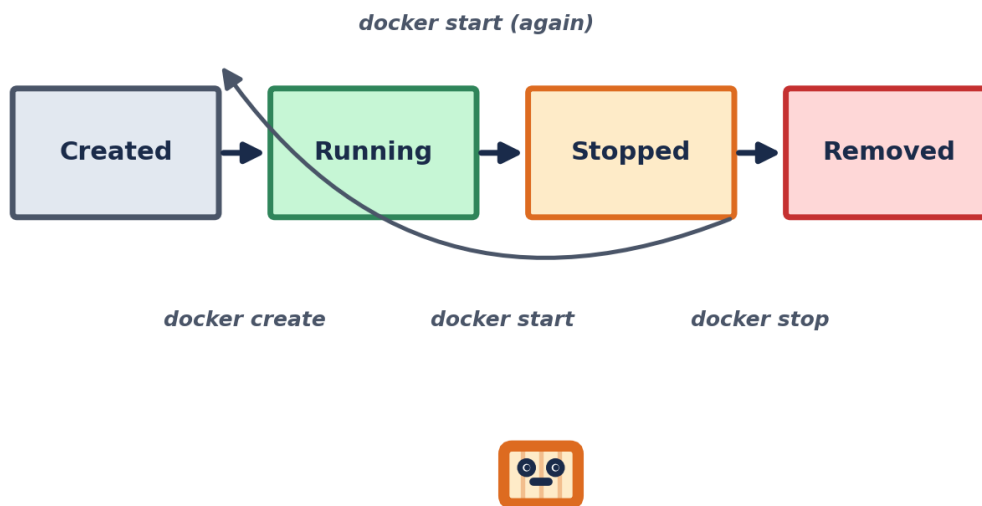
The Container Lifecycle

A container moves through a small, predictable set of states: **Created** (exists, hasn't started), **Running**, **Stopped** (exited, but not yet removed — its writable layer and metadata still exist), and **Removed** (gone for good).

`docker run` is really just `docker create` plus `docker start` in one convenient step. And a stopped container isn't dead — you can `docker start` it again, and it picks up right where it left off, writable layer intact.

Only `docker rm` (or `docker run --rm`'s automatic cleanup) actually destroys that writable layer for good.

The Container Lifecycle



IN SHORT: Stopping a container $\neq$ deleting it. A stopped container can be restarted with its state intact — removal is the one truly destructive step.

CHAPTER 6

docker stop vs. docker kill

docker stop sends the container's main process a **SIGTERM** — a polite request to shut down. The process gets a grace period (10 seconds by default) to finish in-flight work, close connections, and exit cleanly. Only if it hasn't exited by then does Docker escalate to a forceful **SIGKILL**.

docker kill skips the politeness entirely and sends SIGKILL immediately — the process is terminated on the spot, with zero opportunity to clean up.

For this to actually work, your application needs to *handle* SIGTERM — catching the signal and shutting down gracefully. An application that ignores SIGTERM entirely will just sit there for the full grace period before getting forcefully killed anyway.

docker stop vs. docker kill

docker stop



SIGTERM sent.
Grace period to finish
requests, then exit cleanly.

docker kill



SIGKILL sent immediately.
No cleanup, no grace period
-- process just stops.

IN SHORT: Always prefer docker stop over docker kill in normal operation — it gives your application a real chance to shut down cleanly. Reserve kill for a genuinely unresponsive container.

CHAPTER 7

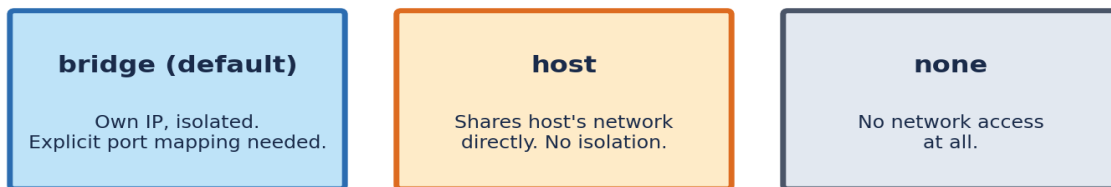
Choosing a Network Mode

By default, every container runs in **bridge** mode — it gets its own private IP address on a virtual network, isolated from the host. To reach it from outside, you explicitly publish a port (-p 8080:80).

host mode drops that isolation — the container shares the host's network stack directly. Faster (no virtual bridge overhead), but the container's ports are the host's ports, with no separation between them.

none mode gives a container no network access at all — useful for workloads that genuinely have no business talking to anything over a network.

Docker Networking Modes



IN SHORT: *bridge is the sensible default for almost everything. Reach for host only when you genuinely need maximum network performance and can accept losing isolation.*

CHAPTER 8

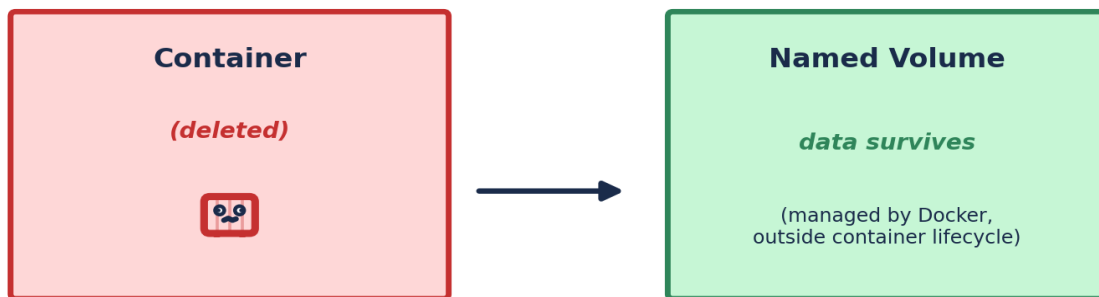
Volumes Outlive Containers

A container's own writable layer disappears the moment the container is removed. If your application writes important data only to that layer, deleting the container deletes that data — permanently, with no undo.

A **volume** is storage that Docker manages independently of any single container's lifecycle. You mount it into a container at a specific path; anything written there persists even after that container is destroyed, and a brand-new container can mount the same volume to pick up right where the old one left off.

This is the fundamental reason databases, uploaded files, and anything else you can't afford to lose should always live in a volume — never just in a container's own ephemeral writable layer.

Volumes Outlive Containers



*Anything written only to the container's own layer is gone when it's removed.
Data that matters belongs in a volume.*

IN SHORT: Treat containers as disposable. Treat volumes as the actual home for anything you can't afford to lose.

CHAPTER 9

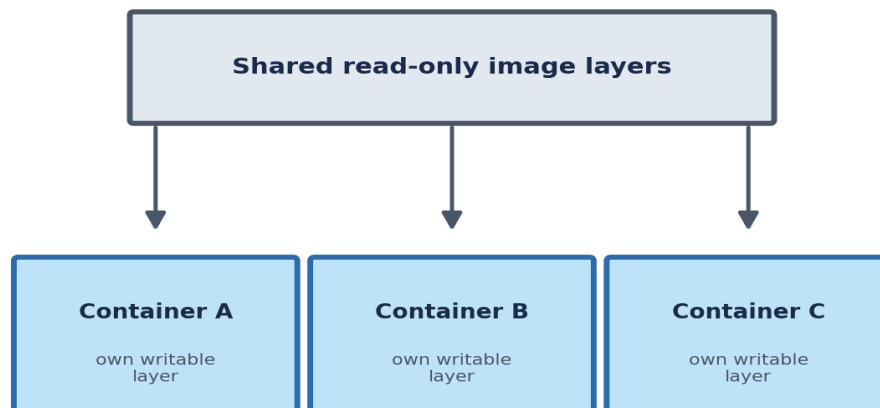
Copy-on-Write: Sharing Without Sharing

Multiple running containers built from the same image all share the exact same underlying read-only image layers in memory and on disk — Docker doesn't duplicate that data per container. This is a big part of why containers are so lightweight.

So what happens when a container modifies a file that originally came from one of those shared layers? Docker uses **copy-on-write**: the file is first copied up into that specific container's own writable layer, and the modification happens on that copy. The original, shared file in the read-only layer is never touched.

This is exactly what keeps containers safe to share layers with each other — no matter how many containers modify their own copy of a shared file, the original layer (and every other container using it) is completely unaffected.

Copy-on-Write: Shared Layers



A write only copies the affected file into that container's own layer -- the shared base layers are never touched.

IN SHORT: Reading is free and shared. The first write to any file triggers a private copy — which is also why write-heavy workloads on large files are better placed on a dedicated volume than left in the container's own layer.

CHAPTER 10

Cheat Sheet

One page, the commands you'll actually reach for day to day. Pin this one.

DOCKER CHEAT SHEET



Images

```
docker build -t name .
```

```
docker images
```

```
docker rmi <image>
```

```
docker pull / push
```

Containers

```
docker run -d -p 8080:80 img
```

```
docker ps -a
```

```
docker stop / start
```

```
docker rm <container>
```

Debugging

```
docker logs -f <container>
```

```
docker exec -it <c> bash
```

```
docker inspect <c>
```

```
docker stats
```

Volumes & Networks

```
docker volume create name
```

```
docker network create name
```

```
docker volume ls
```

```
docker network ls
```

Cleanup

```
docker system prune
```

```
docker image prune
```

```
docker container prune
```